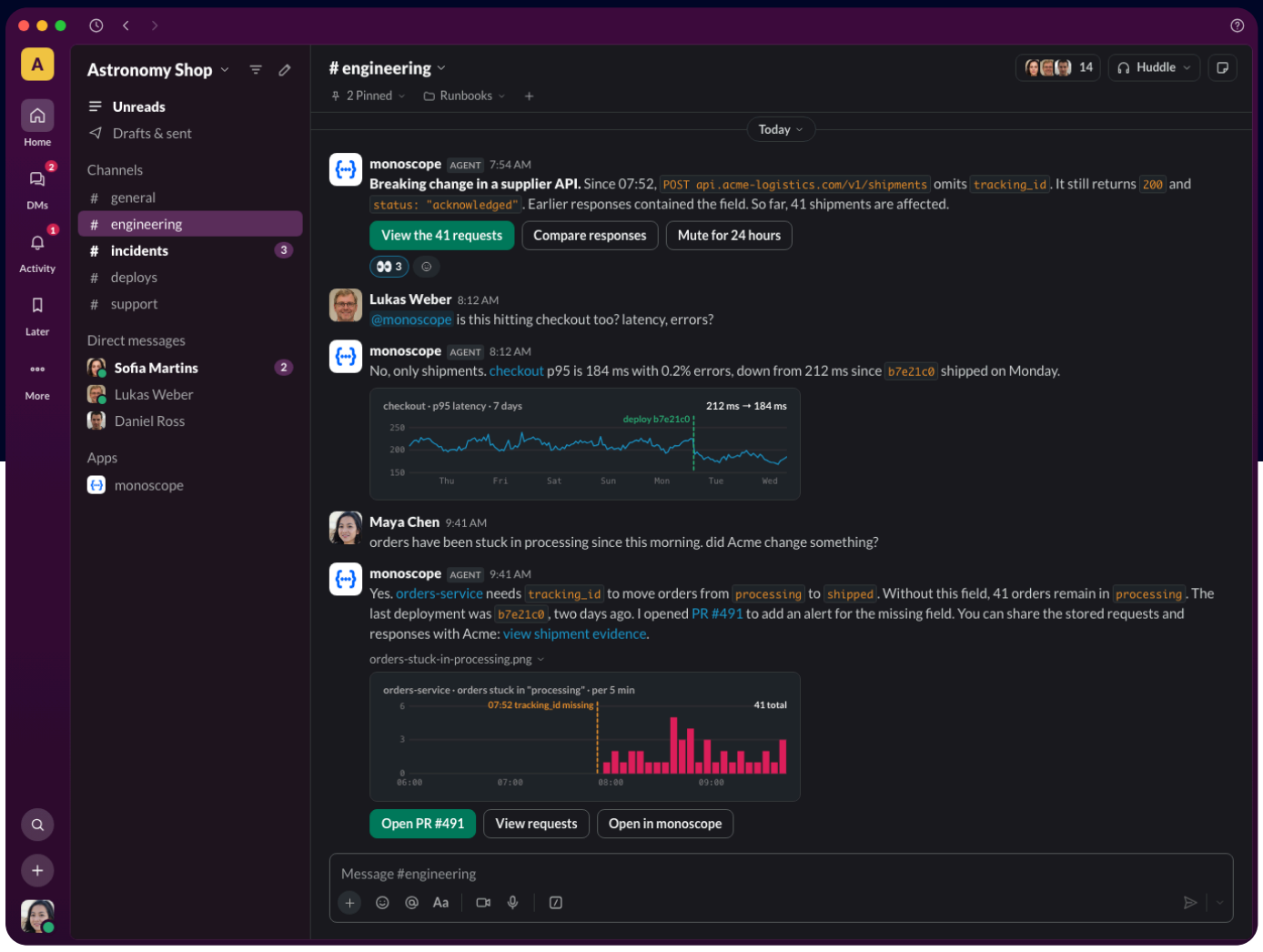
Know what happened. Prove it. Fix it.

Every issue comes with the exact request behind it, and everything around it: the response, the trace, what the customer saw. An AI SRE watches all of it, catches issues as they happen and opens the fix.



07:52

A logistics supplier quietly stops sending `tracking_id`, but its API keeps returning **200 OK**, so nothing looks wrong.

07:54

monoscope spots the breaking change two minutes later and posts it in Slack, with all 41 affected requests attached.

09:41

When an engineer asks what's going on, monoscope answers from the stored requests and opens PR #491.

andela Platnova PayFonte Grovepay SAMEDAY CUSTOM thepoor woodcore Blockradar

07:52 · The incident from the cover

A 200 OK that stranded 41 orders

Acme stopped sending `tracking_id` but kept answering 200 OK, so a typical trace shows a fast, successful call. Without the response body, nothing explains why 41 orders are stuck.

What a typical trace keeps

`POST /v1/shipments` `200` `182 ms`

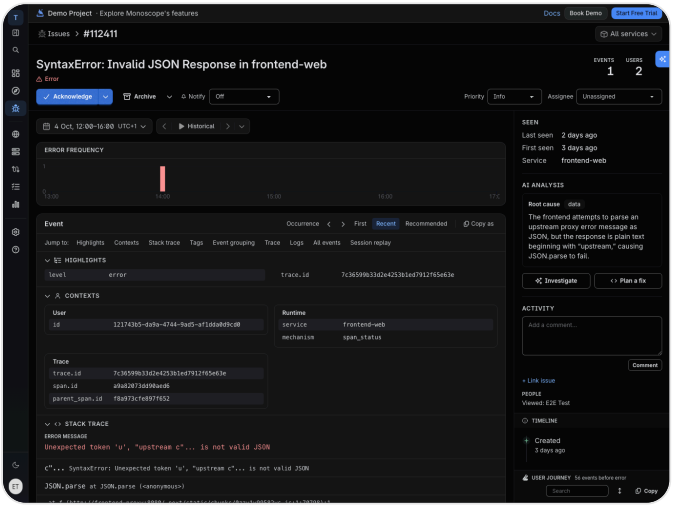
- ✕ No request body
- ✕ No response body
- ✕ Possibly not this request at all, if it was sampled out
- ✕ Nothing you can send the supplier as proof

What monoscope keeps for every request

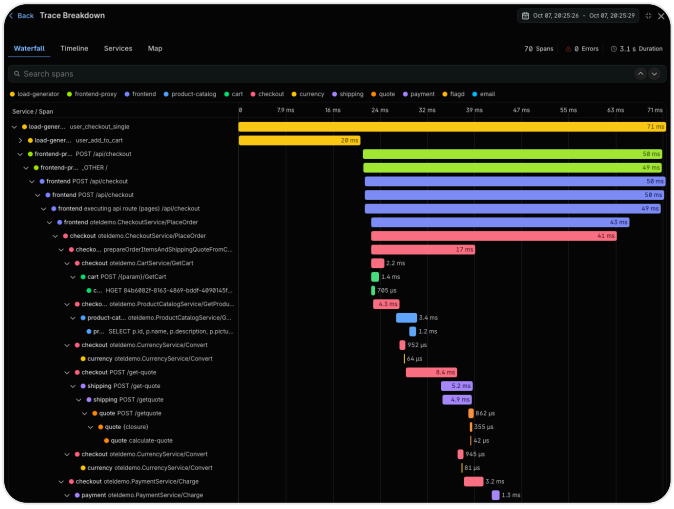
`Response from api.acme-logistics.com at 07:52:14.208`

```
HTTP/1.1 200 OK
{
  "shipment_id": "shp_7Hk2m9",
  "status": "acknowledged",
  "tracking_id": "AX-99812-NG",   missing since 07:52
  "eta": "2026-10-09"
}
```

monoscope notices the missing field, flags it as a breaking change and links it to the trace, the logs and the customer's session.



Related errors are grouped into one issue, with the evidence and an AI analysis already attached.



From the issue, you're one click from the full trace and every service the request touched.

Never sampled

monoscope keeps every request, response, log and span you send, so you never have to guess which events were worth keeping.

Evidence you can share

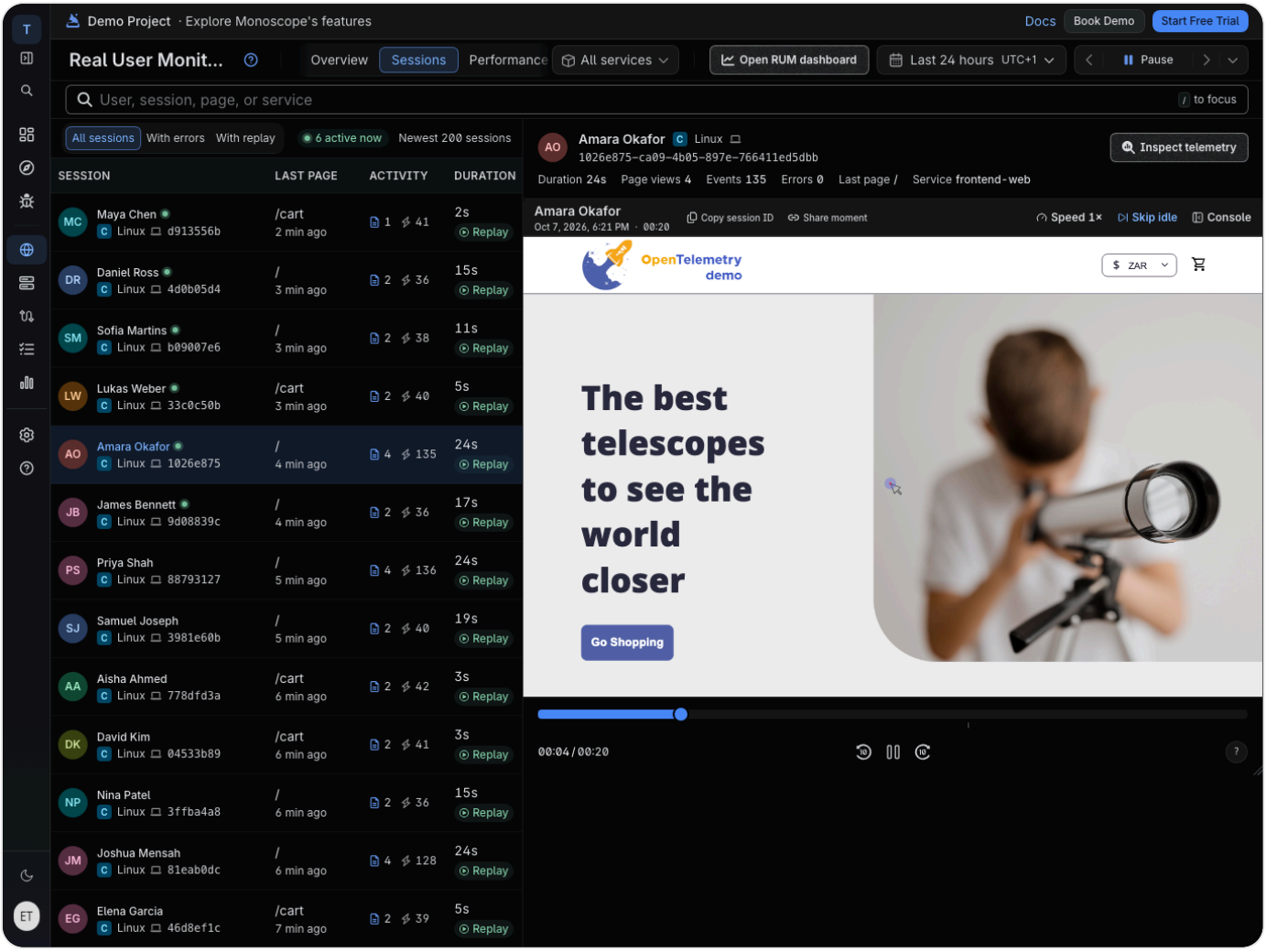
You can see exactly what you sent, what the supplier sent back and when, down to the millisecond, and share it all with a link.

Your bucket is the database

On Cloud + your own S3, your data lives in your own bucket as open Delta Lake tables, and you can keep it for as long as you like.

Some issues are easier to see than explain.

Watch a session replay of what the user saw and did when an issue occurred. Then inspect the request, response and trace.



Watch the user's screen. Pause the replay, inspect their actions and open the session's telemetry. Recorded demo session from the running product.

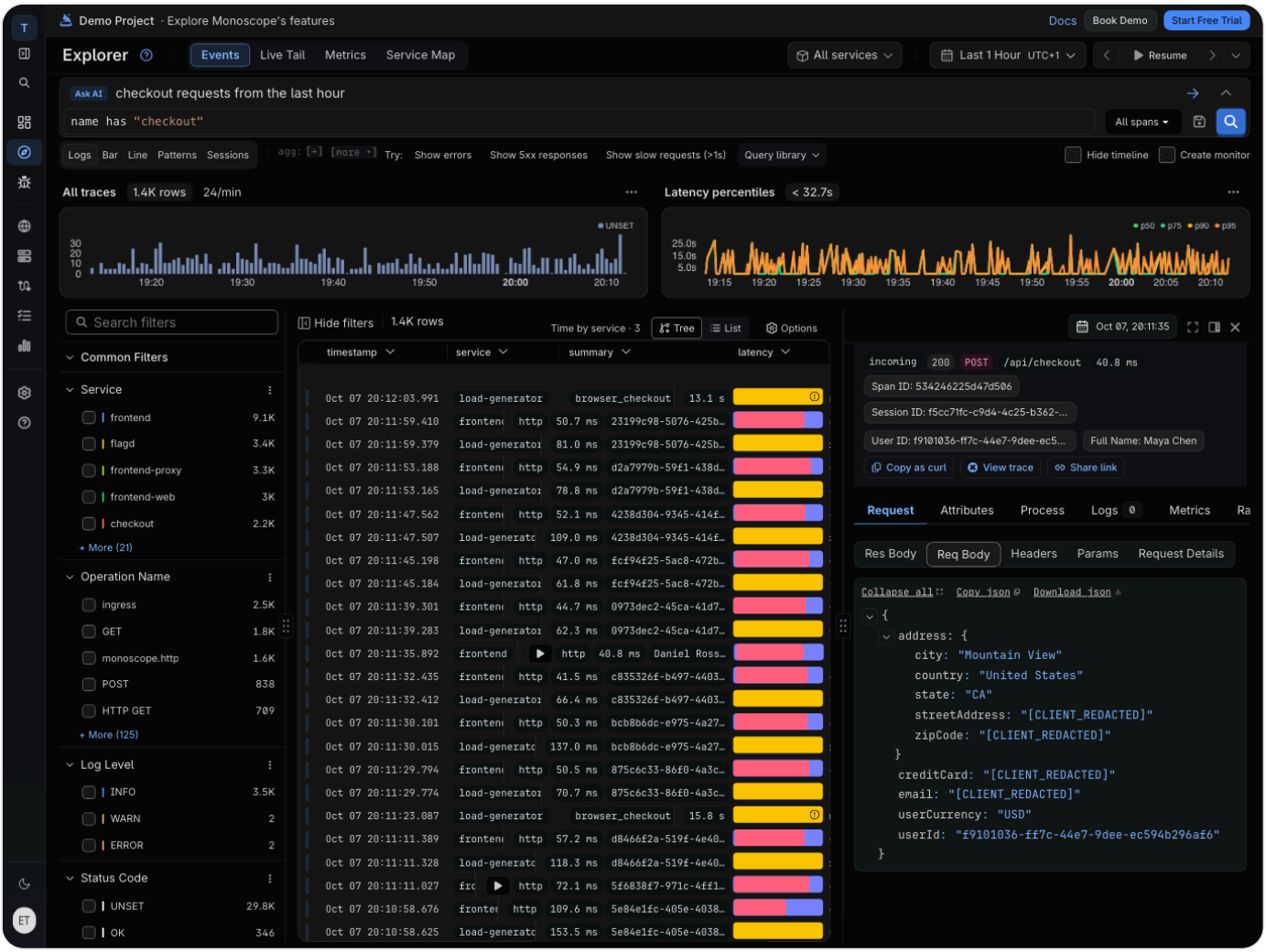
See the sequence

Watch the user's actions and the screens they saw. Pause or replay the moment you need to inspect.

Connect it to the evidence

Open the session's telemetry to inspect the requests and traces behind the user's experience.

What your app sent, and what came back.



Ask in plain language, then read the request. monoscope writes the query and shows it. The body your app sent is there, with sensitive fields redacted.

Read both bodies

A status code tells only part of the story. Open the request and the response, stored in full.

Find the missing field

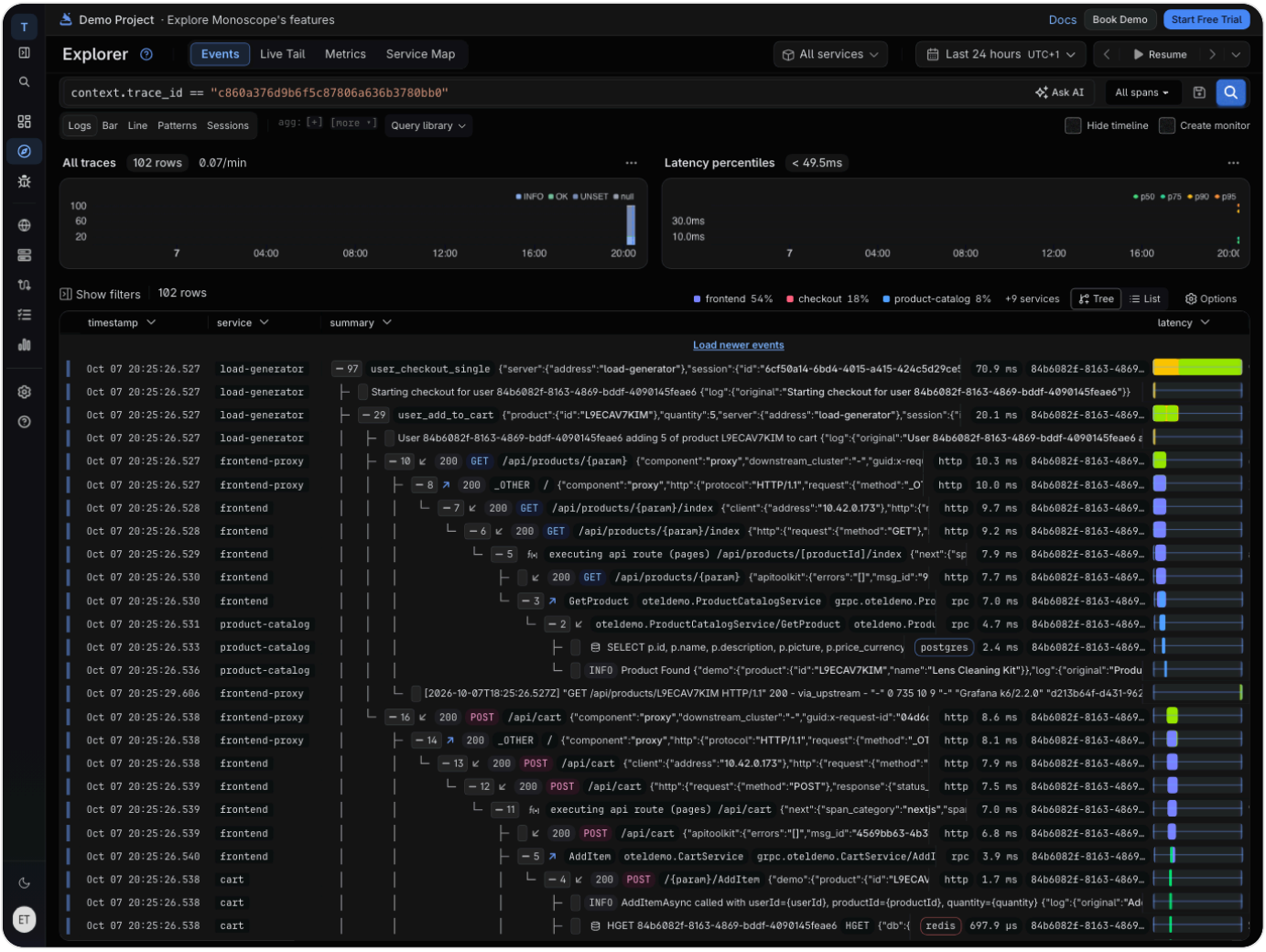
Compare the response with what your app expected, and see why a 200 still caused an issue.

Share the evidence

Send the stored request and response to your team or the supplier with a link.

One checkout, everything it touched.

Open a trace in the explorer and every span and log nests under the request that started it: HTTP calls, gRPC, SQL, Redis and the logs in between.



102 events from one checkout, across 12 services, in the order they happened.

Spans and logs together

Logs sit under the span that wrote them, so you read them in context, not in a separate tool.

Every query and cache call

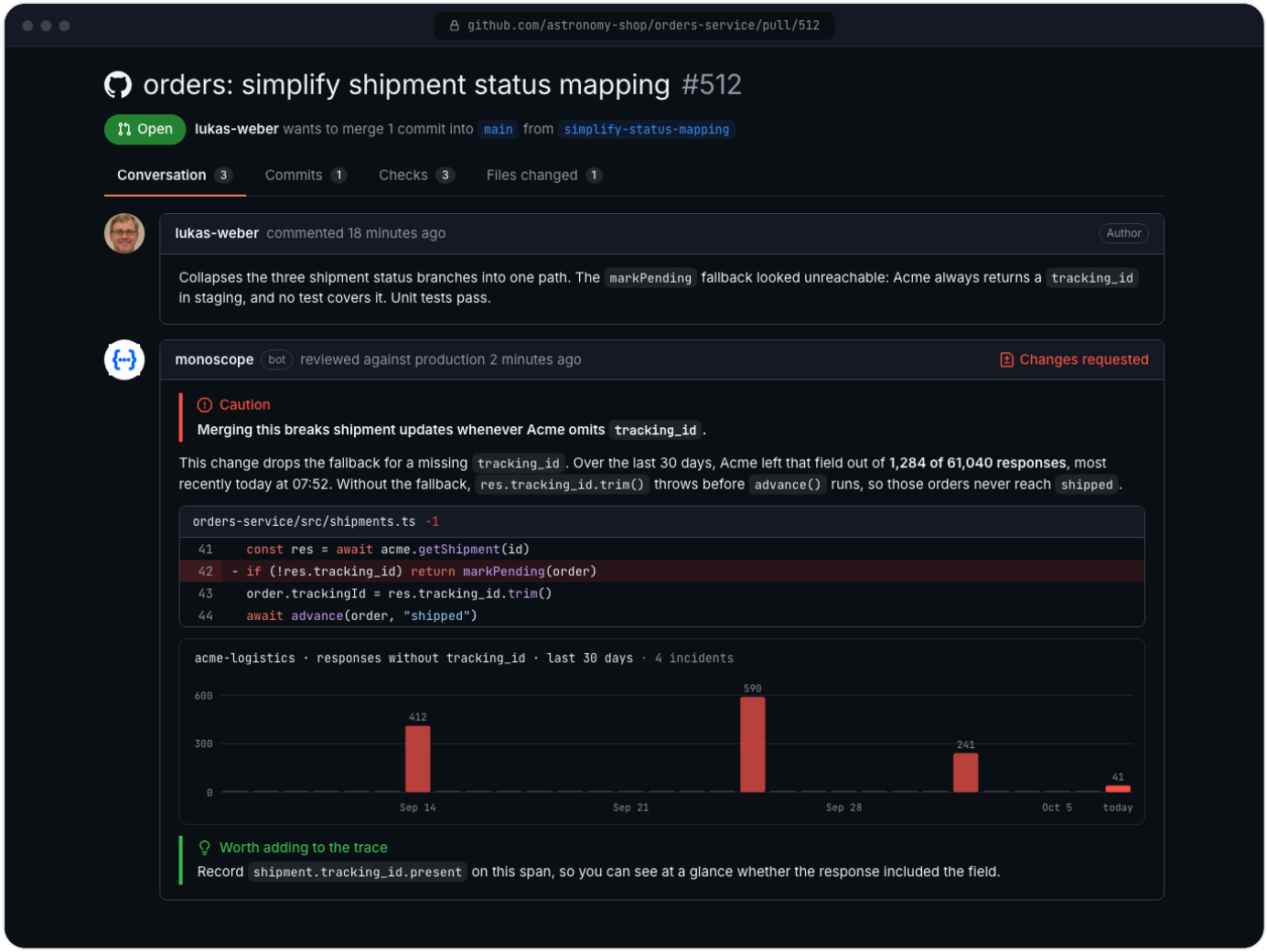
See the SQL a service ran and the Redis keys it read, with timings for each.

Nothing sampled out

monoscope keeps every event, so the trace you need is complete when you open it.

Find the next incident before you merge.

monoscope reviews every pull request, from engineers or coding agents, against your live production data, and shows the evidence when a change looks risky.



Answers in Slack

Ask why checkout is slower, and get an answer from your logs, traces and deploys.

Remembers past issues

Recurring errors keep their history, so each investigation builds on the last.

Finds monitoring gaps

monoscope points out endpoints nobody's watching and ranks alerts by impact.

Works with your agents

Plug it into your own agents through a hosted MCP server with about 50 tools, an agent-mode CLI or Claude Code skills.

Shows its evidence

Every finding links back to the requests, traces and deployments behind it, so you can check the reasoning before you merge.

Reads the full request

The AI works from the full request and response, along with past events and your deployment timeline, not from an error message alone.

Ask Claude Code what broke, and it reads production to find out

If your team already works with Claude Code, Cursor or another coding agent, monoscope plugs straight in. Our open-source skills teach the agent to investigate with the monoscope CLI, so it reads the stored requests, traces and logs instead of guessing from the code.

```
Claude Code  ~/code/acme-shop

> investigate why orders have been stuck in processing since this morning

● Skill(monoscope-skills:investigate)
  └ Successfully loaded skill

● Bash(monoscope facets resource.service.name --top 3)
  └ orders-service 41,208 · checkout-api 18,733 · payments 9,410

● Bash(monoscope logs search 'body has "tracking_id"' --since 6h --first --id-only)
  └ evt_01JA7Q3M9X4K2R

● Bash(monoscope events context --at 2026-10-07T07:52:14Z --window 10m --summary)
  └ 41 traces · orders-service → api.acme-logistics.com · 41 errors in advance()

● Orders stopped advancing at 07:52, and it isn't your code. Acme's POST /v1/shipments still returns 200,
but the response no longer includes tracking_id, so advance() throws in orders-service and 41 orders are
stuck in processing. Your last deploy was b7e21c0, two days ago.

  I can add a fallback that keeps these orders pending, plus an alert for when the field goes missing.
  Want me to open a pull request?
```

An example session with the investigate skill. Each step is a real monoscope CLI command.

The monoscope CLI

One binary that gives your terminal, your CI and your agents the same view of production, with stable JSON output for scripts.

```
curl https://monoscope.tech/install.sh | sh
```

Claude Code skills

Skills for investigating incidents, on-call triage, managing monitors and dashboards, and adding instrumentation.

```
claude plugin install monoscope-skills@monoscope-skills
```

A hosted MCP server

About 50 tools for Claude Desktop, Cursor or your own agents, using the same API key, with nothing to install.

```
https://api.monoscope.tech/api/v1/mcp
```

Send OpenTelemetry, or let your agent set it up

monoscope takes OTLP from any OpenTelemetry SDK or collector. Our open-source skill lets your coding agent set it up and prove the data arrived.

```
Coding agent  ~/code/acme-shop

> instrument this project with OpenTelemetry and send it to monoscope

● Skill(monoscope-skills:instrument)

● Read(package.json)
  └ Node.js · express · undici · pg · kafkajs · bullmq

● Bash(npm install @opentelemetry/sdk-node @opentelemetry/auto-instrumentations-node
@opentelemetry/instrumentation-bullmq)

● Bash(docker run -p 4317:4317 otel/opentelemetry-collector-contrib)
  └ local collector with debug exporter · 6 routes and 2 jobs triggered · every expected span found

● Bash(monoscope traces search 'resource.service.name == "orders-service"' --since 5m)
  └ ✓ service registered · ✓ server spans · ✓ 6/6 routes · ✓ 2/2 jobs · ✓ logs flowing

● orders-service is instrumented and sending to monoscope:

  Incoming HTTP   express   auto-instrumented   ✓
  Outgoing HTTP   undici    auto-instrumented   ✓
  SQL             pg        auto-instrumented   ✓
  Message queues  kafkajs   auto-instrumented   ✓
  Background jobs bullmq    auto-instrumented   ✓
```

An example session on a Node.js service. The skill runs in Claude Code, Cursor, Cline, GitHub Copilot and more.

1 Plans first

Detects your language, framework and libraries, then writes a todo list you can redirect before it edits anything.

2 Covers five surfaces

Incoming and outgoing HTTP, SQL, message queues and background jobs, with official auto-instrumentation where it exists.

3 Checks locally

Runs a local collector, calls your routes and jobs, and confirms each expected span shows up.

4 Confirms in monoscope

Queries monoscope with the CLI and reports pass or fail per route and job. A failed check is never reported as done.

With your coding agent

Add the skills and the CLI, then ask your agent to instrument the project. It checks its work with the CLI.

```
npx skills add monoscope-tech/skills
curl https://monoscope.tech/install.sh | sh
```

By hand, in any language

Point any OpenTelemetry exporter at monoscope. Native SDKs for Node.js, Python, Go, PHP, Java, .NET and Elixir also capture bodies, with redaction.

```
OTEL_EXPORTER_OTLP_ENDPOINT="http://otelcol.monoscope.tech:4317"
OTEL_EXPORTER_OTLP_PROTOCOL="grpc"
OTEL_SERVICE_NAME="orders-service"
OTEL_RESOURCE_ATTRIBUTES="x-api-key=YOUR_API_KEY"
```

What our customers have seen

20x faster mean time to resolution

70% less manual troubleshooting

9x fewer customer support calls

5x faster incident debugging

3x faster system recovery

500ms less response time on improved endpoints

Blockradar · SAMEDAY CUSTOM · Platnova · CORONA MANAGEMENT SYSTEMS · partna · PayFonte

"We had a major incident, and our tech support could see via [monoscope] which third-party integration partner was responsible, and could take action without needing the engineering team's help."

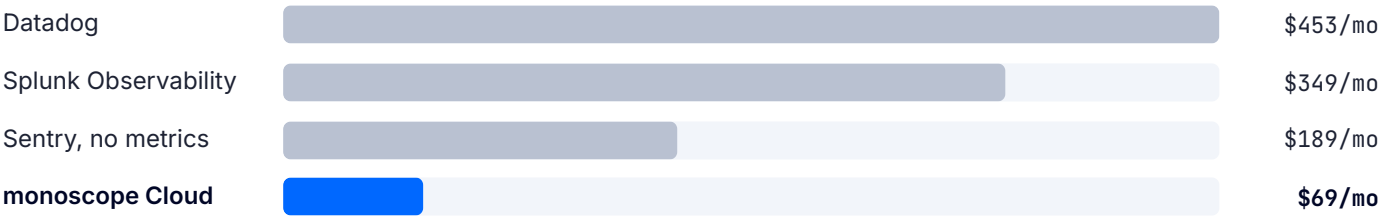
Joshua Chinemezum, CEO, Platnova

"The best observability tool we use today at Woodcore, monoscope notifies us about any slight change that happens on the system."

Samuel Joseph, CEO, Woodcore

Pay per event, not per host

One rate for requests, logs and spans, with separate allowances for metrics and session replay. No per-host fees, so adding servers won't change your bill.



Based on an example workload of 50M events a month, 5 hosts, 2,000 metric series and 5,000 replay sessions, using September 2026 list prices and 30-day retention. You'll find the full assumptions on page 10.

Switching takes three steps

1. **Send your telemetry**
   Point your OpenTelemetry exporter at monoscope, add our SDK, or let your coding agent set it up. We support 780+ integrations.
2. **Run it side by side**
   Keep your current tool running for a week or two and compare which errors and API changes each one catches.
3. **Move your services**
   Once you're happy, send the rest of your services over and retire the tools you don't need anymore.

See it on your own traffic

Start free with up to 10,000 events a day, explore the playground without signing up, or book a call and one of our engineers will help you get set up.

Start for free
no credit card

Try the playground
no signup needed

Book a call
30 minutes

Deployment, security and pricing details

Deployment

Cloud
Stored in monoscope-managed storage, with 30-day retention.

Cloud + your own S3
Stored in your S3-compatible bucket as open Delta Lake and Parquet tables, kept as long as you like.

Self-hosted
Stored on your servers, with retention set by you.

Security

GDPR
We're happy to sign a data processing agreement (DPA) whenever you need one.

EU data region
If you'd prefer, we can store your data in the EU.

SSO
Single sign-on and advanced authentication come with Enterprise.

Redaction
The SDK strips out the headers and body fields you choose, such as `$.password`, before any data leaves your app.

Data ownership
On Cloud + your own S3, your bucket is the database, so you can query it directly with DuckDB, Spark or SQL.

Open source
The whole platform is open source on GitHub under AGPL-3.0, so your team can read the code that handles your data.

Pricing

Cloud
Free up to 10,000 events/day. Paid Cloud: $29/month for 20M events, then $1 per extra million.

Cloud + your own S3
From $199/month for 100M events. Additional events cost $1 per million.

Self-hosted
Community Edition is free (AGPL-3.0). Enterprise starts at $500/month.

Startup program
If you're an eligible startup, we'll cover up to $10,000 of monoscope in your first year.

Volume pricing
Sending more than about a billion events a month? Email hello@monoscope.tech for custom pricing.

Your own numbers
Try the pricing calculator on our site.

Cost estimate on page 9

We based these estimates on September 2026 public list prices, with annual billing where it's offered and 30-day retention for every vendor.

Workload: 50M events/month, 5 hosts, 2,000 metric series and 5,000 replay sessions. Each event averages 2 kB. Each metric series sends one data point per minute.

Datadog: Infrastructure costs $15/host and APM costs $31/host. Log ingestion costs $0.10/GB. Indexed logs cost $2.50/million. Custom metrics cost $5/100 series beyond 100/host. Per 1,000 sessions, RUM costs $0.15 and Session Replay costs $2.50.

Splunk: Observability Cloud App & Infrastructure costs $60/host. Splunk Cloud log ingestion costs about $12.50/month per GB of daily ingestion. RUM costs $14/10,000 sessions. This estimate excludes custom metrics.

Sentry: Team costs $26/month. Spans cost $1.60/million beyond 5M. Errors cost $0.29/1,000, with an assumed error rate of 0.5%. Replays cost $3.75/1,000. This estimate excludes metrics.

monoscope Cloud: $29/month includes 20M events, 20M metric data points and 2,000 replay sessions. Additional usage costs $1/million events, $1/10M metric data points and $1/1,000 replay sessions.